Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- Regular Expressions
- Words
- Corpora
- Simple Unix Tools for Word Tokenization
- Word Tokenization
- Word Normalization, Lemmatization and Stemming
- Sentence Segmentation
- Minimum Edit Distance
- Context Free Grammar

Natural Language processing

- NLP is field of computer science and AI that concerned with computer linguistics and interaction between human and computer natural language.

- Application
- Text processing and analysis
- Text to speech , speech to text, speech to speech
- Machine translation
- Search engine.
- Sentiment analysis and opinion mining.
- Advanced text editor and IDE
- Question answering
- Spam and fraud detection

Regular expression

- Regular expression (often shortened to regex), a language for specifying text search strings.

- Application
- Search in text, pattern in text, string matching, linux terminal
- E.g. Ctrl+F

Pattern	Example	Description
[0-9]	18524	Match only numeric digit
{A-Z}	AZIZ	Only capital word, If word = ‘Aziz’ then not matching
[A-Za-z]	Aziz Ahmed	Capital and small letter but not digit such as Aziz98
[Hmm*]	Hm, Hmm, Hmmm,Hmm.	0 or more matching

Regular expression

Regex	Match (single characters)	Example Patterns Matched
/[^A-Z]/	not an upper case letter	“Oyfn pripetchik”
/[^Ss]/	neither ‘S’ nor ‘s’	“I have no exquisite reason for’t”
/[^.]/	not a period	“our resident Djinn”
/[e^]/	either ‘e’ or ‘^’	“look up ^ now”
/a^b/	the pattern ‘a^b’	“look up a^ b now”

Figure 2.4 The caret ^ for negation or just to mean ^. See below re: the backslash for escaping the period.

Regex	Match	Example Patterns Matched
/woodchucks?/	woodchuck or woodchucks	“woodchuck”
/colou?r/	color or colour	“color”

Figure 2.5 The question mark ? marks optionality of the previous expression.

Regular expression: Kleene closure

- The Kleene closure, also known as the Kleene star, is a unary operator denoted by an asterisk (*) symbol. It is used to indicate that the preceding element or subexpression can occur zero or more times.

- For example, if we have a regular expression "a*", it means that the character 'a' can occur zero or more times. This would match strings such as "", "a", "aa", "aaa", and so on.

- Similarly, in the regular expression "(ab)*", the Kleene closure applies to the entire subexpression "(ab)", indicating that the sequence "ab" can occur zero or more times. This would match strings such as "", "ab", "abab", "ababab", and so on.

Regular expression: Kleene closure

- Zero or more character recognition in a text.
- Application:

  Linux terminal

classification :

"0" = zero or more

"1" = 1 or more

Example 1: New(s)* = New, News, Newss, Newss, Newssss

Example 2: New(s)+ = News, Newsss, Newssss

Regular expression: Kleene closure

- Sometimes it's annoying to have to write the regular expression for digits twice, so there is a shorter way to specify "at least one" of some character.

- One very important special character is the period (/./), a **wildcard** expression that matches any single character (*except* a carriage return), as shown in Fig. 2.6.

Regex	Match	Example Matches
/beg.n/	any character between *beg* and *n*	begin, beg'n, begun

Figure 2.6 The use of the period . to specify any character.

Regular expression: Kleene closure

- **Anchors** are special characters that anchor regular expressions to particular places in a string. The most common anchors are the caret ^ and the dollar sign $. The caret ^ matches the start of a line.

Regex	Match
^	start of line
$	end of line
\b	word boundary
\B	non-word boundary

Figure 2.7 Anchors in regular expressions.

Disjunction, Grouping, and Precedence

- This idea that one operator may take precedence over another, requiring us to sometimes use parentheses to specify what we mean, is formalized by the operator precedence hierarchy for regular expressions.

Parenthesis	()
Counters	* + ? {}
Sequences and anchors	the ^my end$
Disjunction	|

Regular expression: pipe symbol (|)

works similar like logical OR

Choose a particular sub string from the input string

- Go (es|ing): Goes, Going
- S (a|u|i) ng: Sing, Sung, Sang
- Example: Two string

More Operators

Regex	Expansion	Match	First Matches
\d	[0-9]	any digit	Party␣of␣5
\D	[^0-9]	any non-digit	Blue␣moon
\w	[a-zA-Z0-9_]	any alphanumeric/underscore	Daiyu
\W	[^\w]	a non-alphanumeric	!!!!
\s	[␣\r\t\n\f]	whitespace (space, tab)	in_Concord
\S	[^\s]	Non-whitespace	in␣Concord

Figure 2.8 Aliases for common sets of characters.

Regex	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	zero or one occurrence of the previous char or expression
{n}	exactly *n* occurrences of the previous char or expression
{n,m}	from *n* to *m* occurrences of the previous char or expression
{n,}	at least *n* occurrences of the previous char or expression
{,m}	up to *m* occurrences of the previous char or expression

Figure 2.9 Regular expression operators for counting.

Regular expression

Regex	Match	First Patterns Matched
*	an asterisk "*"	"K*A*P*L*A*N"
\.	a period "."	"Dr. Livingston, I presume"
\?	a question mark	"Why don't they come and lend a hand?"
\n	a newline	
\t	a tab	

Figure 2.10 Some characters that need to be backslashed.

Words

- A computer-readable **corpora** collection of text or speech. (plural corpus)

- For example the Brown corpus is a million-word collection of samples from 500 written English texts from different genres (newspaper, fiction, non-fiction, academic, etc.), assembled at Brown University in 1963–64

Words

Corpus	Instances = N	Types = $\|V\|$
Shakespeare	884 thousand	31 thousand
Brown corpus	1 million	38 thousand
Switchboard telephone conversations	2.4 million	20 thousand
COCA	440 million	2 million
Google n-grams	1 trillion	13 million

Figure 2.11 Rough numbers of wordform types and instances for some English language corpora. The largest, the Google n-grams corpus, contains 13 million types, but this count only includes types appearing 40 or more times, so the true number would be much larger.

Text normalization in NLP

Normalization is the process of converting into standard form.

Necessity of normalization
Word boundary detection
Separated word from each other
Example: Uzbekistan, New York, cats and dogs
Example 2: #nlp

The world has 7097 languages. -2018

Tokenization

- There are roughly two classes of tokenization algorithms. In top-down tokenization, we define a standard and implement rules to implement that kind of tokenization. In bottom-up tokenization, we use simple statistics of letter sequences to break up words into subword tokens.

- While the Unix command sequence just removed all the numbers and punctuation, for most NLP applications we'll need to keep these in our tokenization. We often want to break off punctuation as a separate token; commas are a useful piece of information for parsers, periods help indicate sentence boundaries.

Methods of Text Normalization:

- Tokenization

  Input: Uzbekistan is beautiful

  Output: [Uzbekistan, is, beautiful]

Lemmatization: Finding the same root.

Input: [Sings, sung, sang], [connection, connected, connecting]

Output: [Sing], [connect]

A **lemma** is a set of lexical forms having the same stem, the same major part-of-speech, and the same word sense

Methods of Text Normalization:

- Stemming: porter stemmer

Rules 1 : ational -> ate, ator -> ate

Example 1: Relational -> Relate, Operator -> Operate

Rule 2: Ing -> Null, s-> Null

Output: Going -> Go, Walking -> Walk, Cats -> Cat

Stemming Challenge : Need to handle some exceptional case

Input: King, Nothing, News

Output: K, noth, New

Methods of Text Normalization:

- Stop words: Remove the common word that are not important
- words: am is are, an the
- Example:

  Input: Tashkent has a beautiful park.

  Output: Tashkent beautiful park

- Parts Of Speech Tagging: Identify the grammar of each word

  Input: IUT has a beautiful campus

  Output: IUT = Noun, has = verb, a = Article, beautiful = adjective , campus = Noun.

Tokenization

- Tokenization includes word, sentence, whitespace, punctuation.

```
[1]  import nltk
```

```
[4]  nltk.version_file
     nltk.__author__
```

```
'NLTK Team'
```

```
[5]  nltk.download("punkt")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
True
```

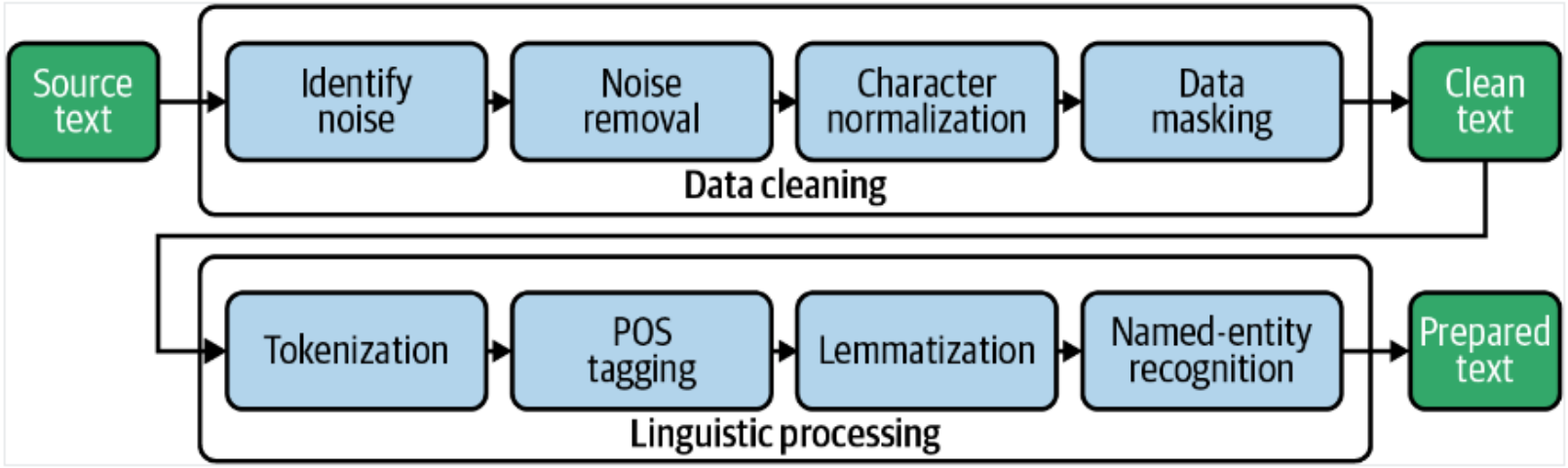
Sentence Tokenization

```
[6]  from nltk.tokenize import sent_tokenize
```

Sentence Tokenization

Tokenization is a text preprocessing step in sentiment analysis that involves breaking down the text into individual words or tokens.

Preprocessing Text



Word tokenization

Word Tokenization

0s

```
from nltk.tokenize import word_tokenize
tokenized_word=word_tokenize(text)
word_tokenize(text)
```

```
['Department',
 'of',
 'Computer',
 'Science',
 'and',
 'Engineering',
 'of',
 'IUT',
 ',',
 'started',
 'its',
 'academic',
 'activities',
 'from',
 '2014',
 'with',
```

Text Normalization: Case folding

- All letter to lower case
- Application

  -Information extraction, sentiment analysis, machine translation

- Example

  Input

  . [CAT, I have an Apple, General Motors, US]

  Output

  . [cat, i have an apple, general motors, us]

Minimum Edit distance

- Spell correction

  Cornel

  Korenel

  Corrnel

  Cononel

- Bioinformatics

  sample 1 : [A,T,G,G,C]

  sample 2 : [A,T,C,G,C]

Applications

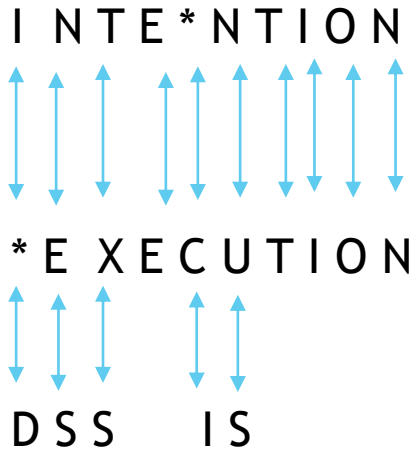
- Search
- Machine translation
- Information extraction
- Speech recognition

Minimum Edit Distance

- The minimum number of editing operations need to be transform one string to another.

- INTENTION
- EXECUTION

- Editing Operation
  - Insertion, I
  - Deletion, D
  - Substitution, S



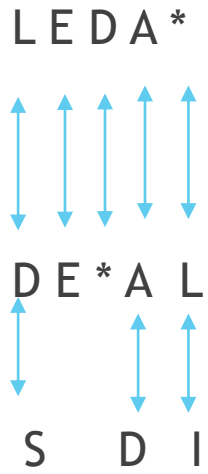
Minimum Edit Distance

- Operation cost
- I = 1
- D = 1
- S = (D+I)=2

- Cost = 1+2+2+1 +2 = 8

Minimum Edit Distance

- Ambiguity of minimum edit distance calculation



Cost= 2+2+2 =6

cost = 2+1+1=4

Dynamic programming for Minimum edit distance

- **Dynamic programming**: A tabular computation of D(n,m)
- Solving problems by combining solutions to subproblems.
- Bottom-up
  - We compute D(i,j) for small i,j
  - And compute larger D(i,j) based on previously computed smaller values
  - i.e., compute D(i,j) for all i ($0 < i < n$) and j ($0 < j < m$)

Defining Minimum edit distance (Levenshtein)

- Initialization

  $D(i,0) = i$

  $D(0,j) = j$

- Recurrence Relation:

  For each $i = 1 \ldots M$

  For each $j = 1 \ldots N$

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

- Termination:

  $D(N,M)$ is distance

The Edit distance table

Destination(j)

Source (i)

	0	1	2	3	4	5
0		#	L	E	D	A
1	#	0	1	2	3	4
2	D	1				
3	E	2				
4	A	3				
5	L	4				

Initialization

D(i,0) = i
D (0,j) = j

The Edit distance table

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

Destination(j)

Source (i)	0	1	2	3	4	5
		#	L	E	D	A
0	#	0	1	2	3	4
1	D	1	2	3		
2	E	2				
3	A	3				
4	L	4				
5						

Initialization

i = 2 (index value: D)
j = 2 (index value: L)

D(i,j)= min{D(1,2)+1 , D(2,1)+1, D(1,1)+2}
= min{2,2,2} = 2

The Edit distance table

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

Destination(j)

Source (i)	0	1	2	3	4	5
		#	L	E	D	A
0	#	0	1	2	3	4
1	D	1	2	3		
2	E	2				
3	A	3				
4	L	4				
5						

D(i,j)= min{D(1,2)+1 , D(2,1)+1, D(1,1)+2}
= min{2,2,2} = 2

i= 2(index value: D)
j= 3(index value: E)

D(i,j)= min {D(1,3)+1 , D(2,2)+1, D(1,2)+2}
= min {3,3,3} = 3

The Edit distance table

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } X(i) \neq Y(j) \\ 0; & \text{if } X(i) = Y(j) \end{cases} \end{cases}$$

Destination(j)

Source (i)	0	1	2	3	4	5
		#	L	E	D	A
0	#	0	1	2	3	4
1	D	1	2	3	2	3
2	E	2	3	2	3	4
3	A	3	4	3	4	3
4	L	4	3	4	5	4
5						

i= 5(index value: L)
j= 5(index value: A)

D(i,j)= min {D(4,5)+1 , D(5,4)+1, D(4,4)+2}
= min {4,6,6} = 4

DP for the minimum Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

Edit Distance

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 \\ D(i,j-1) + 1 \\ D(i-1,j-1) + \begin{cases} 2; & \text{if } S_1(i) \neq S_2(j) \\ 0; & \text{if } S_1(i) = S_2(j) \end{cases} \end{cases}$$

N	9									
O	8									
I	7									
T	6									
N	5									
E	4									
T	3									
N	2									
I	1									
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

The Edit Distance Table

N	9	8	9	10	11	12	11	10	9	**8**
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	**0**	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

Context Free Grammar (CFG)

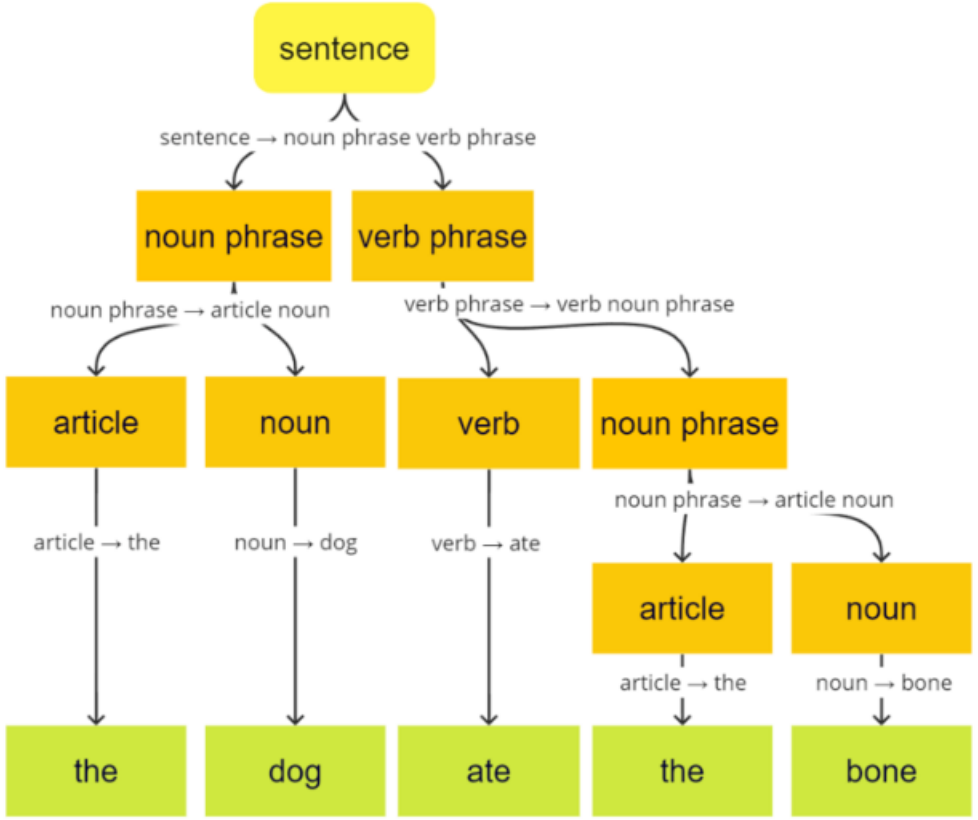
- Context free grammar is a formal grammar which is used to generate all possible strings in a given formal language.

Non-Terminal Symbols = {sentence, noun, verb, article, noun phrase, verb phrase}

Terminal Symbols = {dog, bone, the, ate}

Production Rules = {sentence → noun phrase verb phrase,
noun phrase → article noun,
verb phrase → verb noun phrase,
noun → dog or bone,
verb → ate,
article → the}

Start Symbol = sentence



Advantages of CFG

- There are the various usefulness of CFG:
- Context free grammar is useful to describe most of the programming languages.
- If the grammar is properly designed then an efficient parser can be constructed automatically.
- Using the features of associatively & precedence information, suitable grammars for expressions can be constructed.
- Context free grammar is capable of describing nested structures like: balanced parentheses, matching begin-end, corresponding if-then-else's & so on.

Context Free Grammar (CFG)

- A CFG consists of a finite set of grammar rules is a quadruple (N,T,P,S)

- N is a set of non terminal symbol
- T is a set of terminal symbol
- P is a set of rules
- S is the start symbol

S-> Aa , A-> Ab|c

CFG

Example:

- N = {S,A}
- T = {a, b, c }
- P = {S-> Aa, A-> Ab, A-> c}
- S = S

Context Free Grammar (CFG)

- **Production rules:**

$S \rightarrow aSa$

$S \rightarrow bSb$

$S \rightarrow c$

check that abbcbba string can be derived from the given CFG.

$S \Rightarrow aSa$
$S \Rightarrow abSba$
$S \Rightarrow abbSbba$
$S \Rightarrow abbcbba$

Categories of CFG

- Non Recursive CFG

  S -> Aa output:{ba, ca} S -> Aa
  -> ba

  A-> b|c

- Recursive CFG

  S -> Aa output: { ca, cba, cbba, cbbba, …. } S-> Aa
  ->Aba
  -> cba

  A -> Ab|c

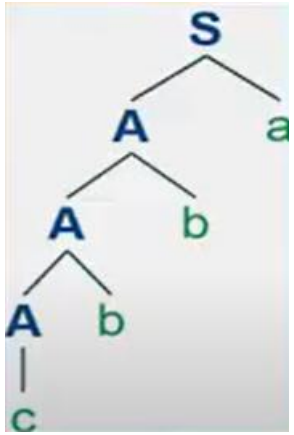
Context Free Grammar (CFG)

Example of a CFG
S -> Aa
A -> Ab| c

Input: {c, b, b, a}

S-> Aa
-> Aba
-> Abba
-> cbba

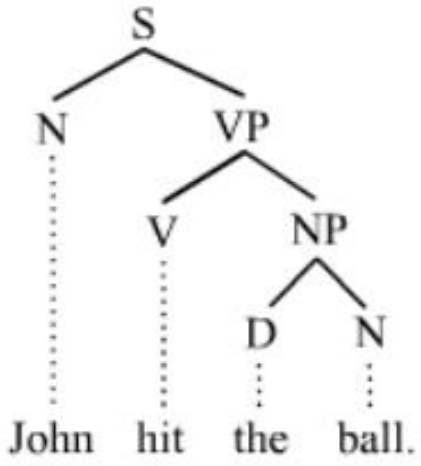
Parsing using CFG



Parse tree

Parse tree

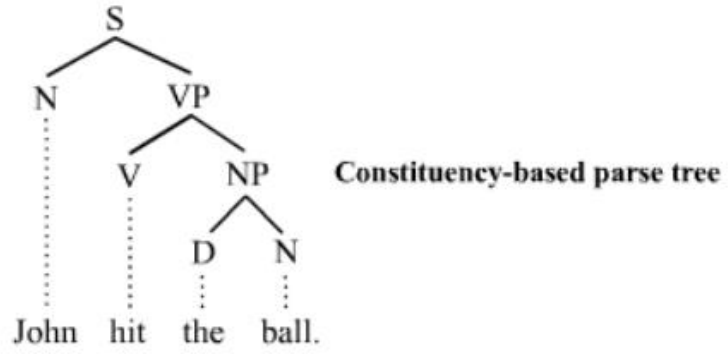
- A parse tree or parsing tree or derivation tree or concrete syntax tree is an ordered, rooted tree that represents the syntactic structure of a string according to some context-free grammar. The term parse tree itself is used primarily in computational linguistics;



Constituency-based parse tree

Parse tree

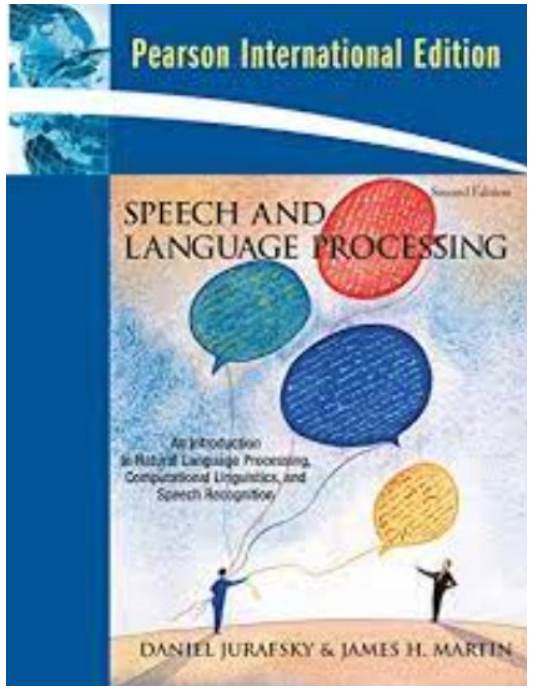
- The constituency-based parse trees of constituency grammars (phrase structure grammars) distinguish between terminal and non-terminal nodes. The interior nodes are labeled by non-terminal categories of the grammar, while the leaf nodes are labeled by terminal categories.



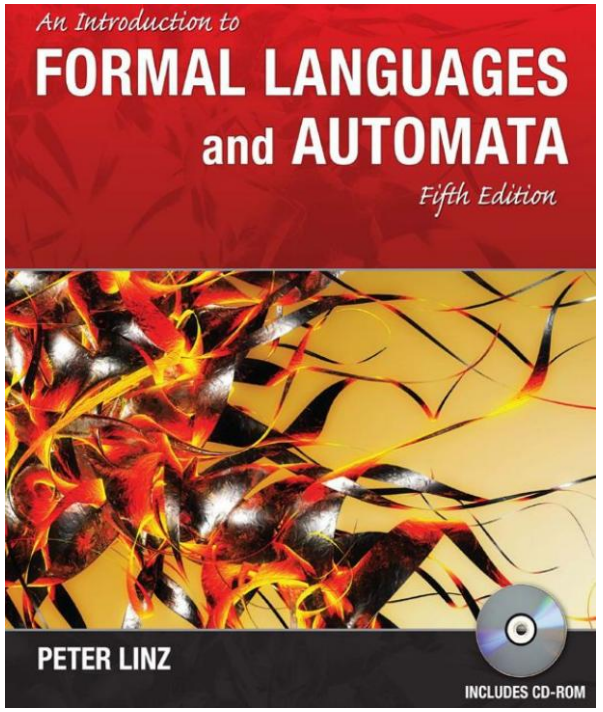
Constituency-based parse tree

S for sentence, the top-level structure in this example
NP for noun phrase. The first (leftmost) NP, a single noun "John", serves as the subject of the sentence.
The second one is the object of the sentence.
VP for verb phrase, which serves as the predicate
V for verb. In this case, it's a transitive verb *hit*.
D for determiner, in this instance the definite article "the“ N for noun

Reference



Chapter 2



Chapter 5

Question

Thank you